

Answer Set Programming for Planning

WASP Relational Learning Course 2026

Jordan Thayer

May 18, 2026

Machine Reasoning Lab
Linköping University

1. **ASP fundamentals.** You can read clingo code by the end.
2. **Re-encode Dominik's SAT theory in ASP.** Same theory, different formalism.
3. **ASP for dead-end detection.** The project; what you'll write in the notebook.

Part 1 has three short hands-on segments in the lecture notebook (`asp-fundamentals.ipynb`).

Clone this repo if you didn't already: <https://github.com/mrlab-ai/wasp-asp-fundamentals>

You'll need clingo to run the code snippets: <https://potassco.org/clingo/> (or via apt, etc.)

Part 1: ASP Fundamentals

Why ASP?

- Dominik just showed you SAT for generalized policies.
- ASP is YACL (Yet Another Constraint Language).

Why ASP?

- Dominik just showed you SAT for generalized policies.
- ASP is YACL (Yet Another Constraint Language).
- ASP is more expressive than SAT, having native constructs for
 - combinatorial generation
 - counting
 - optimization
- We'll use ASP to learn dead-end classifiers in the project

Why ASP?

- Dominik just showed you SAT for generalized policies.
- ASP is YACL (Yet Another Constraint Language).
- ASP is more expressive than SAT, having native constructs for
 - combinatorial generation
 - counting
 - optimization
- We'll use ASP to learn dead-end classifiers in the project
 - but first you need to be able to read it.

Why ASP?

- Dominik just showed you SAT for generalized policies.
- ASP is YACL (Yet Another Constraint Language).
- ASP is more expressive than SAT, having native constructs for
 - combinatorial generation
 - counting
 - optimization
- We'll use ASP to learn dead-end classifiers in the project
 - but first you need to be able to read it.

One-liner

ASP = Prolog-style rules + SAT-style search + native aggregates and optimization.

What exactly is an Answer Set?

An **answer set** is a set of atoms — a candidate description of what's true.

To qualify as an answer set of a program, it must:

What exactly is an Answer Set?

An **answer set** is a set of atoms — a candidate description of what's true.

To qualify as an answer set of a program, it must:

- **Satisfy every rule.** If the body holds, the head holds.

What exactly is an Answer Set?

An **answer set** is a set of atoms — a candidate description of what's true.

To qualify as an answer set of a program, it must:

- **Satisfy every rule.** If the body holds, the head holds.
- **Be supported.** Every atom in it traces back to facts via rules. No “free-floating” truth.

What exactly is an Answer Set?

An **answer set** is a set of atoms — a candidate description of what's true.

To qualify as an answer set of a program, it must:

- **Satisfy every rule.** If the body holds, the head holds.
- **Be supported.** Every atom in it traces back to facts via rules. No “free-floating” truth.
- **Be minimal.** Nothing extra. Atoms not derivable from the program are taken as false (closed-world assumption).

What exactly is an Answer Set?

An **answer set** is a set of atoms — a candidate description of what's true.

To qualify as an answer set of a program, it must:

- **Satisfy every rule.** If the body holds, the head holds.
- **Be supported.** Every atom in it traces back to facts via rules. No “free-floating” truth.
- **Be minimal.** Nothing extra. Atoms not derivable from the program are taken as false (closed-world assumption).

Solving an ASP program means **enumerating its answer sets**. A program may have zero, one, or many.

What exactly is an Answer Set?

An **answer set** is a set of atoms — a candidate description of what's true.

To qualify as an answer set of a program, it must:

- **Satisfy every rule.** If the body holds, the head holds.
- **Be supported.** Every atom in it traces back to facts via rules. No “free-floating” truth.
- **Be minimal.** Nothing extra. Atoms not derivable from the program are taken as false (closed-world assumption).

Solving an ASP program means **enumerating its answer sets**. A program may have zero, one, or many.

Contrast with SAT

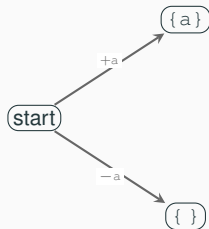
A SAT model can assign any unconstrained variable however it likes. An answer set cannot: every true atom needs a justification from the program. “Consistent with the formula” vs. “forced by the program.”

Where Do Answer Sets Come From?

Conceptually, the solver builds a **search tree**. Each branch is a partial guess; rules propagate within a branch; integrity constraints prune entire branches. The surviving leaves are the answer sets.

Tiny program:

- a may be in or out.
- b may be in or out.
- c is in whenever a is.
- a and b cannot both be in.

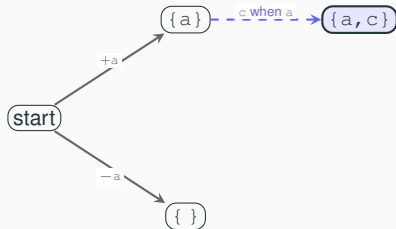


Where Do Answer Sets Come From?

Conceptually, the solver builds a **search tree**. Each branch is a partial guess; rules propagate within a branch; integrity constraints prune entire branches. The surviving leaves are the answer sets.

Tiny program:

- a may be in or out.
- b may be in or out.
- c is in whenever a is.
- a and b cannot both be in.

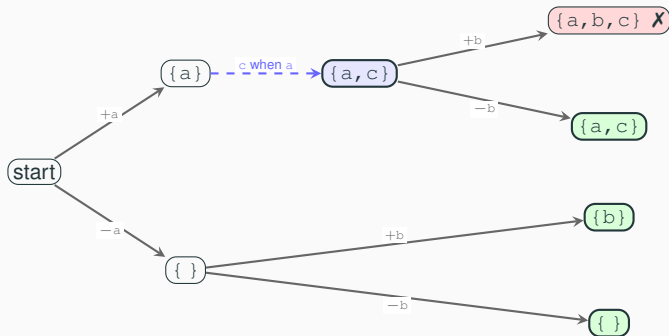


Where Do Answer Sets Come From?

Conceptually, the solver builds a **search tree**. Each branch is a partial guess; rules propagate within a branch; integrity constraints prune entire branches. The surviving leaves are the answer sets.

Tiny program:

- a may be in or out.
- b may be in or out.
- c is in whenever a is.
- a and b cannot both be in.



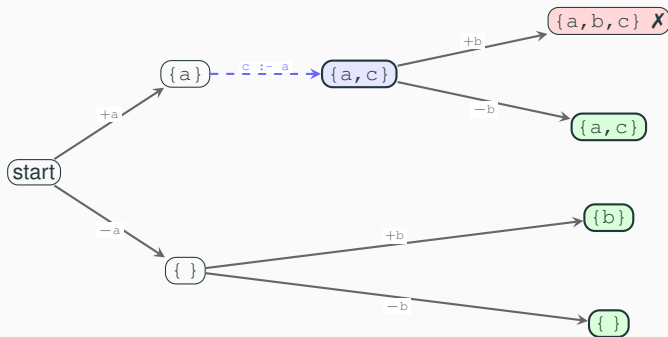
Three branches survive — three answer sets: $\{a, c\}$, $\{b\}$, $\{\}$. The dashed step shows the rule firing inside the $+a$ branch; the fourth candidate is pruned by the constraint.

Where Do Answer Sets Come From? (in clingo)

Same picture, now with the actual clingo program. You haven't seen the syntax yet — just notice the shape; we unpack each construct over the next few slides.

Program:

```
{ a }.  
{ b }.  
c :- a.  
:- a, b.  
  
{ a }. — choice.  
c :- a. — rule.  
:- a, b. — constraint.
```



Three answer sets: $\{a, c\}$, $\{b\}$, $\{\}$. The dashed step is $c :- a$ firing inside the $+a$ branch; $:- a, b.$ prunes the fourth candidate.

The ASP Mental Model: Generate / Constrain / Optimize

G

C

O

If you remember one thing from this lecture, remember GCO. Most non-trivial ASP programs follow this pattern.

The ASP Mental Model: Generate / Constrain / Optimize

Generate enumerate candidate solutions
(choice rules — coming in a few slides).

Constrain filter out the bad ones
(integrity constraints).

Optimize prefer the best of what's left
(`#minimize` / `#maximize`).

If you remember one thing from this lecture, remember GCO. Most non-trivial ASP programs follow this pattern.

Before we get there: what's a rule?

Facts and Rules

A clingo program is a set of **facts** and **rules**.

Fact: an atom asserted to be true.

Facts and Rules

A clingo program is a set of **facts** and **rules**.

Fact: an atom asserted to be true.

```
sunny.
```

```
warm.
```

Facts and Rules

A clingo program is a set of **facts** and **rules**.

Fact: an atom asserted to be true.

```
sunny.
```

```
warm.
```

Rule: `head :- body.`

“If body holds, head does too.”

Facts and Rules

A clingo program is a set of **facts** and **rules**.

Fact: an atom asserted to be true.

```
sunny.  
warm.
```

Rule: `head :- body.`

“If body holds, head does too.”

```
go_outside :- sunny, warm.  
\pause
```

Multiple rules with the same head are an implicit disjunction:

Facts and Rules

A clingo program is a set of **facts** and **rules**.

Fact: an atom asserted to be true.

```
sunny.  
warm.
```

Rule: `head :- body.`

“If body holds, head does too.”

```
go_outside :- sunny, warm.  
\pause
```

Multiple rules with the same head are an implicit disjunction:

```
stay_inside :- raining.  
stay_inside :- snowing.
```

Facts and Rules

A clingo program is a set of **facts** and **rules**.

Fact: an atom asserted to be true.

```
sunny.  
warm.
```

Rule: head :- body.

“If body holds, head does too.”

```
go_outside :- sunny, warm.  
\pause
```

Multiple rules with the same head are an implicit disjunction:

```
stay_inside :- raining.  
stay_inside :- snowing.  
  
stay_inside :- new_videogame_released.  
stay_inside :- salesman_outside.
```

Facts and Rules

A clingo program is a set of **facts** and **rules**.

Fact: an atom asserted to be true.

```
sunny.  
warm.
```

Rule: head :- body.

“If body holds, head does too.”

```
go_outside :- sunny, warm.  
\pause
```

Multiple rules with the same head are an implicit disjunction:

```
stay_inside :- raining.  
stay_inside :- snowing.  
  
stay_inside :- new_videogame_released.  
stay_inside :- salesman_outside.
```

Same idea as Prolog or Datalog. ASP-specific things start in two slides.

Facts and Rules

A clingo program is a set of **facts** and **rules**.

Fact: an atom asserted to be true.

```
sunny.  
warm.
```

Rule: `head :- body.`

“If body holds, head does too.”

```
go_outside :- sunny, warm.  
\pause
```

Multiple rules with the same head are an implicit disjunction:

```
stay_inside :- raining.  
stay_inside :- snowing.  
  
stay_inside :- new_videogame_released.  
stay_inside :- salesman_outside.
```

Same idea as Prolog or Datalog. ASP-specific things start in two slides.

Try it

You'll need clingo to run the code snippets: <https://potassco.org/clingo/> (or via apt, etc.)
`asp-fundamentals.ipynb`, Section 1.

Variables and Grounding

Rules can use **variables** (uppercase names). The same rule applies to every consistent assignment.

Variables and Grounding

Rules can use **variables** (uppercase names). The same rule applies to every consistent assignment.

```
ball(b1) . ball(b2) .  
in(b1, room_a) .
```

Variables and Grounding

Rules can use **variables** (uppercase names). The same rule applies to every consistent assignment.

```
ball(b1). ball(b2).  
in(b1, room_a).
```

```
at(B, R) :- ball(B), in(B, R).
```

Before search, clingo **grounds** the program: substitute variables with values from the facts. For the example:

Variables and Grounding

Rules can use **variables** (uppercase names). The same rule applies to every consistent assignment.

```
ball(b1). ball(b2).
```

```
in(b1, room_a).
```

```
at(B, R) :- ball(B), in(B, R).
```

Before search, clingo **grounds** the program: substitute variables with values from the facts. For the example:

```
at(b1, room_a) :- ball(b1), in(b1, room_a).    % body holds; derived
```

```
at(b2, room_a) :- ball(b2), in(b2, room_a).    % body fails; nothing
```

Answer set: `at(b1, room_a).` *Ground first, then solve* — different from Prolog, which interleaves the two.

Variables and Grounding

Rules can use **variables** (uppercase names). The same rule applies to every consistent assignment.

```
ball(b1). ball(b2).
```

```
in(b1, room_a).
```

```
at(B, R) :- ball(B), in(B, R).
```

Before search, clingo **grounds** the program: substitute variables with values from the facts. For the example:

```
at(b1, room_a) :- ball(b1), in(b1, room_a).    % body holds; derived
```

```
at(b2, room_a) :- ball(b2), in(b2, room_a).    % body fails; nothing
```

Answer set: `at(b1, room_a).` *Ground first, then solve* — different from Prolog, which interleaves the two.

Try it

`asp-fundamentals.ipynb`, Section 2.

Choice Rules

So far every rule has been deterministic: if the body holds, the head is true.

Choice Rules

So far every rule has been deterministic: if the body holds, the head is true.

A **choice rule** lets the head be optional.

$\{ p(X) \} :- \text{body}.$

Reads: “for each x for which body holds, the answer set MAY contain $p(x).$ ”

Choice Rules

So far every rule has been deterministic: if the body holds, the head is true.

A **choice rule** lets the head be optional.

```
{ p(X) } :- body.
```

Reads: “for each x for which `body` holds, the answer set MAY contain `p(x)`.”

Worked example:

```
item(a). item(b). item(c).  
{ pack(I) } :- item(I).
```

Choice Rules

So far every rule has been deterministic: if the body holds, the head is true.

A **choice rule** lets the head be optional.

```
{ p(X) } :- body.
```

Reads: “for each x for which `body` holds, the answer set MAY contain `p(x)`.”

Worked example:

```
item(a). item(b). item(c).  
{ pack(I) } :- item(I).
```

Eight candidate answer sets, one per subset of $\{a, b, c\}$.

Choice rules are the **Generate** in GCO.

Choice Rules

So far every rule has been deterministic: if the body holds, the head is true.

A **choice rule** lets the head be optional.

```
{ p(X) } :- body.
```

Reads: “for each x for which `body` holds, the answer set MAY contain $p(x)$.”

Worked example:

```
item(a). item(b). item(c).  
{ pack(I) } :- item(I).
```

Eight candidate answer sets, one per subset of $\{a, b, c\}$.

Choice rules are the **Generate** in GCO.

Try it

`asp-fundamentals.ipynb`, Section 3.

Integrity Constraints

- Syntax: `:- bad_thing.` (a rule with no head)

Integrity Constraints

- Syntax: `:- bad_thing.` (a rule with no head)
- Equivalently, think of it as $\perp$ `:- bad_thing.`
- The missing head is implicit $\perp$

Integrity Constraints

- Syntax: `:- bad_thing.` (a rule with no head)
- Equivalently, think of it as $\perp$ `:- bad_thing.`
- The missing head is implicit $\perp$
- no answer set may contain $\perp$, so the body is forbidden.

Integrity Constraints

- Syntax: `:- bad_thing.` (a rule with no head)
- Equivalently, think of it as `⊥ :- bad_thing.`
- The missing head is implicit `⊥`
- no answer set may contain `⊥`, so the body is forbidden.

Reads: “no answer set contains `bad_thing.`”

This is the **Constrain** in GCO;
it filters candidates from a choice rule.

Continuing from the previous slide — forbid packing both `a` and `b`:

```
:- pack(a), pack(b).
```

Integrity Constraints

- Syntax: `:- bad_thing.` (a rule with no head)
- Equivalently, think of it as $\perp$ `:- bad_thing.`
- The missing head is implicit $\perp$
- no answer set may contain $\perp$, so the body is forbidden.

Reads: “no answer set contains `bad_thing.`”

This is the **Constrain** in GCO;
it filters candidates from a choice rule.

Continuing from the previous slide — forbid packing both `a` and `b`:

```
:- pack(a), pack(b).
```

The 8 candidates collapse to 6: those containing both `pack(a)` and `pack(b)` are eliminated.

Aggregates

Pattern. `#op { tuple : conditions }`

build a multiset of `tuple`-values for which `conditions` hold, then apply `#op`.

Pattern. `#op { tuple : conditions }`

build a multiset of `tuple`-values for which `conditions` hold, then apply `#op`.

Count: forbid packing more than 2 items

```
:- #count { I : pack(I) } > 2.
```

Aggregates

Pattern. `#op { tuple : conditions }`

build a multiset of `tuple`-values for which `conditions` hold, then apply `#op`.

Count: forbid packing more than 2 items

```
:- #count { I : pack(I) } > 2.
```

Generate `{ I : pack(I) }` $\rightarrow$ **apply** `#count` $\rightarrow$ **Constrict** when `> 2`.

Pattern. `#op { tuple : conditions }`

build a multiset of `tuple`-values for which `conditions` hold, then apply `#op`.

Count: forbid packing more than 2 items

```
:- #count { I : pack(I) } > 2.
```

Generate `{ I : pack(I) }` $\rightarrow$ **apply** `#count` $\rightarrow$ **Constrict** when `> 2`.

Sum: forbid total weight over capacity

```
:- #sum { W, I : pack(I), weight(I, W) } > capacity.
```

Pattern. `#op { tuple : conditions }`

build a multiset of `tuple`-values for which `conditions` hold, then apply `#op`.

Count: forbid packing more than 2 items

```
:- #count { I : pack(I) } > 2.
```

Generate `{ I : pack(I) }` $\rightarrow$ **apply** `#count` $\rightarrow$ **Constrict** when `> 2`.

Sum: forbid total weight over capacity

```
:- #sum { W, I : pack(I), weight(I, W) } > capacity.
```

Generate `{ (W, I) : pack(I)  $\wedge$  weight(I, W) }` $\rightarrow$ **apply** `#sum` $\rightarrow$ **Constrict** when `> capacity`.

Why does `I` stay in the sum's tuple?

Aggregates

Pattern. `#op { tuple : conditions }`

build a multiset of `tuple`-values for which `conditions` hold, then apply `#op`.

Count: forbid packing more than 2 items

```
:- #count { I : pack(I) } > 2.
```

Generate `{ I : pack(I) }` $\rightarrow$ **apply** `#count` $\rightarrow$ **Constrict** when `> 2`.

Sum: forbid total weight over capacity

```
:- #sum { W, I : pack(I), weight(I, W) } > capacity.
```

Generate `{ (W, I) : pack(I)  $\wedge$  weight(I, W) }` $\rightarrow$ **apply** `#sum` $\rightarrow$ **Constrict** when `> capacity`.

Why does `I` stay in the sum's tuple?

Two items with the same weight would otherwise dedupe to one element and undercount.

```
#maximize { V, I : pack(I), value(I, V) }.  
\pause
```

The V, I part is the **multiset key**: each (V, I) pair contributes V once. The key matters because identical V 's would otherwise collapse in a set semantics.

```
#maximize { V, I : pack(I), value(I, V) }.  
\pause
```

The V, I part is the **multiset key**: each (V, I) pair contributes V once. The key matters because identical V 's would otherwise collapse in a set semantics.

`#minimize` works the same way. Multiple optimization statements layer with priorities.

This is the **Optimize** in GCO.

```
#maximize { V, I : pack(I), value(I, V) }.  
\pause
```

The V, I part is the **multiset key**: each (V, I) pair contributes V once. The key matters because identical V 's would otherwise collapse in a set semantics.

`#minimize` works the same way. Multiple optimization statements layer with priorities.

This is the **Optimize** in GCO.

We'll see a worked example in the full knapsack problem, just hold your horses.

#show and clingo Invocation

```
#show pack/1.
```

Restricts what appears in the printed answer set. Internal predicates (`weight`, `value`, `item facts` and any auxiliaries) get hidden.

Run:

```
$ clingo --opt-mode=optN program.lp
```

- Without `-opt-mode optN`: clingo prints intermediate optima as it improves.
- With `optN`: clingo prints all proven-optimal answer sets.

Output: a flat list of `pack( . . . )` atoms; an optional cost line (`Optimization: 17`); an optimality marker (`OPTIMUM FOUND`).

Putting It Together: Knapsack

```
item(a). item(b). item(c).  
weight(a, 2). weight(b, 3). weight(c, 5).  
value(a, 4). value(b, 5). value(c, 8).  
#const capacity = 6.
```

Putting It Together: Knapsack

```
item(a). item(b). item(c).  
weight(a, 2). weight(b, 3). weight(c, 5).  
value(a, 4). value(b, 5). value(c, 8).  
#const capacity = 6.  
  
{ pack(I) } :- item(I). % Generate
```

Putting It Together: Knapsack

```
item(a). item(b). item(c).  
weight(a, 2). weight(b, 3). weight(c, 5).  
value(a, 4). value(b, 5). value(c, 8).  
#const capacity = 6.  
  
{ pack(I) } :- item(I). % Generate  
  
:- #sum { W, I : pack(I), weight(I, W) } > capacity. % Constrain
```

Putting It Together: Knapsack

```
item(a). item(b). item(c).  
weight(a, 2). weight(b, 3). weight(c, 5).  
value(a, 4). value(b, 5). value(c, 8).  
#const capacity = 6.  
  
{ pack(I) } :- item(I). % Generate  
  
:- #sum { W, I : pack(I), weight(I, W) } > capacity. % Constrain  
  
#maximize { V, I : pack(I), value(I, V) }. % Optimize  
#show pack/1.
```

- Choice rule generates 8 candidates.
- Integrity constraint cuts to 6 (those over weight 6).
- Optimization picks the highest-value feasible subset.

Putting It Together: Knapsack

```
item(a). item(b). item(c).  
weight(a, 2). weight(b, 3). weight(c, 5).  
value(a, 4). value(b, 5). value(c, 8).  
#const capacity = 6.  
  
{ pack(I) } :- item(I). % Generate  
  
:- #sum { W, I : pack(I), weight(I, W) } > capacity. % Constrain  
  
#maximize { V, I : pack(I), value(I, V) }. % Optimize  
#show pack/1.
```

- Choice rule generates 8 candidates.
- Integrity constraint cuts to 6 (those over weight 6).
- Optimization picks the highest-value feasible subset.

We also have N-Queens in the repository if you want more!

Part 2: Re-encode Dominik's SAT Theory in ASP

Recall: SAT Theory for Generalized Policies (Lecture 1 Slide 21 Redux)

Variables

$Select(f)$ feature f is used by the policy.

$Good(s, s')$ transition (s, s') is π -compatible.

Core constraints

1. Every alive state has at least one good outgoing transition.
2. Good and bad transitions must be separable by some selected feature.

Optimization

Minimize total complexity of selected features.

Reference: lecture_1, "Learning Generalized Policies via Satisfiability."

How ASP Models the Generalized-Policy Problem

The bridge

Each **answer set** of the program is one valid generalized policy.

How ASP Models the Generalized-Policy Problem

The bridge

Each **answer set** of the program is one valid generalized policy.

Same shape as SAT, recast in ASP idiom:

- **Variables become choices.** One knob per SAT variable; the answer set records which knobs are on.
- **Clauses become integrity constraints.** They don't add atoms — they prune the candidate answer sets that violate them.
- **Objective becomes #minimize.** Same cost function, native syntax.

How ASP Models the Generalized-Policy Problem

The bridge

Each **answer set** of the program is one valid generalized policy.

Same shape as SAT, recast in ASP idiom:

- **Variables become choices.** One knob per SAT variable; the answer set records which knobs are on.
- **Clauses become integrity constraints.** They don't add atoms — they prune the candidate answer sets that violate them.
- **Objective becomes `#minimize`.** Same cost function, native syntax.

Mental shortcut

SAT model = a policy. Answer set = a policy. Different answer sets are different valid policies, not the same one in disguise; `#minimize` singles out the cheapest.

Variable Mapping (SAT $\rightarrow$ ASP) (Lecture 1 Slide 21 Redux)

SAT	ASP
-----	-----

Variable Mapping (SAT $\rightarrow$ ASP) (Lecture 1 Slide 21 Redux)

SAT	ASP
-----	-----

Select(f)

Variable Mapping (SAT $\rightarrow$ ASP) (Lecture 1 Slide 21 Redux)

SAT

ASP*Select(f)*`{ select(F) } :- feature(F, _).`

Variable Mapping (SAT $\rightarrow$ ASP) (Lecture 1 Slide 21 Redux)

SAT	ASP
$Select(f)$	<code>{ select(F) } :- feature(F, _).</code>
$Good(s, s')$	

Variable Mapping (SAT $\rightarrow$ ASP) (Lecture 1 Slide 21 Redux)

SAT**ASP**

Select(f)`{ select(F) } :- feature(F, _).`*Good(s, s')*`{ good(S1, S2) } :- transition(S1, S2).`

Variable Mapping (SAT $\rightarrow$ ASP) (Lecture 1 Slide 21 Redux)

SAT**ASP**

Select(f)

```
{ select(F) } :- feature(F, _).
```

Good(s, s')

```
{ good(S1, S2) } :- transition(S1, S2).
```

feature facts

Variable Mapping (SAT $\rightarrow$ ASP) (Lecture 1 Slide 21 Redux)

SAT

ASP*Select(f)*

```
{ select(F) } :- feature(F, _).
```

Good(s, s')

```
{ good(S1, S2) } :- transition(S1, S2).
```

feature facts

```
feature(f1, 3).  feature(f2, 5).
```

Variable Mapping (SAT $\rightarrow$ ASP) (Lecture 1 Slide 21 Redux)

SAT

ASP

Select(f)

```
{ select(F) } :- feature(F, _).
```

Good(s, s')

```
{ good(S1, S2) } :- transition(S1, S2).
```

feature facts

```
feature(f1, 3).  feature(f2, 5).
```

transition facts

Variable Mapping (SAT $\rightarrow$ ASP) (Lecture 1 Slide 21 Redux)

SAT

ASP

Select(f)

```
{ select(F) } :- feature(F, _).
```

Good(s, s')

```
{ good(S1, S2) } :- transition(S1, S2).
```

feature facts

```
feature(f1, 3).  feature(f2, 5).
```

transition facts

```
transition(s1, s2).  alive_state(s1).
```

Variable Mapping (SAT $\rightarrow$ ASP) (Lecture 1 Slide 21 Redux)

SAT

ASP

Select(*f*)`{ select(F) } :- feature(F, _).`*Good*(*s*, *s'*)`{ good(S1, S2) } :- transition(S1, S2).`

feature facts

`feature(f1, 3). feature(f2, 5).`

transition facts

`transition(s1, s2). alive_state(s1).` $\Delta_f(s, s')$

Variable Mapping (SAT $\rightarrow$ ASP) (Lecture 1 Slide 21 Redux)

SAT

ASP*Select(f)*`{ select(F) } :- feature(F, _).`*Good(s, s')*`{ good(S1, S2) } :- transition(S1, S2).`

feature facts

`feature(f1, 3). feature(f2, 5).`

transition facts

`transition(s1, s2). alive_state(s1).` $\Delta_f(s, s')$ `observable_change(F, S1, S2, V)`

Variable Mapping (SAT $\rightarrow$ ASP) (Lecture 1 Slide 21 Redux)

SAT**ASP**

Select(f)`{ select(F) } :- feature(F, _).`*Good(s, s')*`{ good(S1, S2) } :- transition(S1, S2).`

feature facts

`feature(f1, 3). feature(f2, 5).`

transition facts

`transition(s1, s2). alive_state(s1).` $\Delta_f(s, s')$ `observable_change(F, S1, S2, V)`*F changes across (S₁, S₂), witnessed by V*

The `_` in `feature(F, _)` is the complexity carried by the fact but irrelevant at choice time.

We pull it back in via `feature(F, C)` when we `#minimize` later.

Constraint 1: Every Alive State Has a Good Outgoing Transition

SAT (per alive s):

$$\bigvee_{\{s' \mid (s, s') \in T\}} \textit{Good}(s, s')$$

ASP:

Constraint 1: Every Alive State Has a Good Outgoing Transition

SAT (per alive s):

$$\bigvee_{\{s' \mid (s, s') \in T\}} \text{Good}(s, s')$$

ASP:

```
:- alive_state(S1), { good(S1, S2) : transition(S1, S2) } = 0.
```

Constraint 1: Every Alive State Has a Good Outgoing Transition

SAT (per alive s):

$$\bigvee_{\{s' \mid (s, s') \in T\}} \text{Good}(s, s')$$

ASP:

```
:- alive_state(S1), { good(S1, S2) : transition(S1, S2) } = 0.
```

Reads: “no answer set has an alive state with zero good outgoing transitions.”

Bridge: SAT’s per-state existential over outgoing transitions becomes an ASP integrity constraint over a counting aggregate.

Constraint 2: Separation by a Selected Feature

SAT:

$$Good(s, s') \wedge \neg Good(t, t') \rightarrow \bigvee_{\substack{f \in \mathcal{F} \\ \Delta_f(s, s') \neq \Delta_f(t, t')}} Select(f)$$

```
:- good(S1, S2), not good(T1, T2),  
   { select(F) : select(F),  
     observable_change(F, S1, S2, V1),  
     observable_change(F, T1, T2, V2),  
     V1 != V2 } = 0.
```

Constraint 2: Separation by a Selected Feature

SAT:

$$Good(s, s') \wedge \neg Good(t, t') \rightarrow \bigvee_{\substack{f \in \mathcal{F} \\ \Delta_f(s, s') \neq \Delta_f(t, t')}} Select(f)$$

```
:- good(S1, S2), not good(T1, T2),  
   { select(F) : select(F),  
     observable_change(F, S1, S2, V1),  
     observable_change(F, T1, T2, V2),  
     V1 != V2 } = 0.
```

- Pick any (good, bad) transition pair.

Constraint 2: Separation by a Selected Feature

SAT:

$$Good(s, s') \wedge \neg Good(t, t') \rightarrow \bigvee_{\substack{f \in \mathcal{F} \\ \Delta_f(s, s') \neq \Delta_f(t, t')}} Select(f)$$

```
:- good(S1, S2), not good(T1, T2),  
   { select(F) : select(F),  
     observable_change(F, S1, S2, V1),  
     observable_change(F, T1, T2, V2),  
     V1 != V2 } = 0.
```

- Pick any (good, bad) transition pair.
- Aggregate: count *selected* features F whose observable change differs across the pair.

Constraint 2: Separation by a Selected Feature

SAT:

$$Good(s, s') \wedge \neg Good(t, t') \rightarrow \bigvee_{\substack{f \in \mathcal{F} \\ \Delta_f(s, s') \neq \Delta_f(t, t')}} Select(f)$$

```
:- good(S1, S2), not good(T1, T2),  
   { select(F) : select(F),  
     observable_change(F, S1, S2, V1),  
     observable_change(F, T1, T2, V2),  
     V1 != V2 } = 0.
```

- Pick any (good, bad) transition pair.
- Aggregate: count *selected* features F whose observable change differs across the pair.
- Count zero $\Rightarrow$ no selected feature distinguishes them $\Rightarrow$ forbidden.

SAT recap

Variables:

$Select(f)$, $Good(s, s')$

Constraints:

- Each alive s : some good outgoing.
- Good/bad pairs separated by selected feature.

Minimize $\sum$ selected feature complexity.

ASP

```
{ select(F) } :- feature(F, _).  
{ good(S1, S2) } :- transition(S1, S2).  
  
:- alive_state(S1),  
   { good(S1, S2) : transition(S1, S2) } = 0.  
  
:- good(S1, S2), not good(T1, T2),  
   { select(F) : select(F),  
     observable_change(F, S1, S2, V1),  
     observable_change(F, T1, T2, V2),  
     V1 != V2 } = 0.  
  
#minimize { C, F : select(F), feature(F, C) }.  
#show select/1.  
#show good/2.
```

Same theory, different formalism.

Part 3: ASP for Dead-End Detection

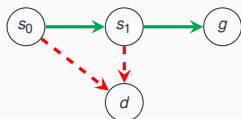
Bridge: Same Shape, Different Problem

Recall the separation constraint from Part 2 (Slide on Constraint 2).

Substitute three things:

General policies	Unsolvability
<code>good(S1, S2)</code>	<code>solvable(S1)</code>
<code>not good(T1, T2)</code>	<code>unsolvable(S2)</code>
<code>observable_change/4</code>	<code>value/3</code>
<i>transitions distinguished</i>	<i>states distinguished</i>

General policy: label transitions



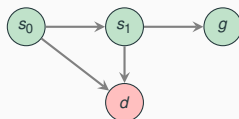
Pivot.

What if instead of separating *good and bad transitions*, we separate *solvable and unsolvable states*?

Same constraint shape, different domain.

That's the dead-end detector.

Unsolvability: label states



The Unsolvability Classifier Setup

What we have (provided)

- Training states labeled solvable or unsolvable (CSV).
- A feature column per state, integer-valued (CSV).
- A feature complexity table (the same `feature/2` facts as before).

What we want

Pick a subset of features whose values distinguish all solvable states from all unsolvable states.

What you write in the notebook

- The ASP separation constraint (over states, not transitions).
- `select/1` is what we want as output.

The Spanner notebook gives you the data, the harness, and the helper code. You write the ASP.

T-Perfect

- One feature set, picked globally.
- Must separate every (solvable, unsolvable) training pair simultaneously.
- Minimize total feature complexity.

Brittle. If no small feature set distinguishes every pair at once, the chosen set must grow or the problem is infeasible.

T-Safe

- One feature set *per unsolvable state*; disjoin them.
- Each per-state problem need only separate one unsolvable from all solvables.
- Each subproblem optimized independently.

Robust. Always feasible as long as each per-state subproblem is; degrades gracefully into a disjunction of small rules.

Same goal — fit the training data — different shape of solution.

Project: Scope and Toolchain

What you write

Just the ASP encodings: T-Perfect (Slide 20) and T-Safe (Slide 22, building on T-Perfect). Everything else is provided.

What's provided in the project notebook

- Spanner training data (feature CSV, labels, complexity table).
- Helper code: `feature/2` and `value/3` fact generation, `learn_t_perfect_model`, `learn_tsafe_model`, scoring utilities.
- Train/valid/test split (Step 2), KNN baseline (Step 3), ML bake-off scaffolding (Step 6).

Toolchain

`clingo` (already used in the lecture-companion notebook), Python, scikit-learn, pandas.

Go forth and exercise! https://github.com/mrlab-ai/wasp_asp_exercise